

1. What is wrapper class? What is their use?

Wrapper classes are used to convert any data type into an object. The primitive data types are not objects; they do not belong to any class; they are defined in the language itself. Sometimes, it is required to convert data types into objects in Java language. The eight classes of the `java.lang` package are known as wrapper classes in Java. The list of eight wrapper classes are given below: Boolean, character, byte, short, integer, long, float, double.

2. Which are methods of `java.lang.Object` class? Which are native methods of object class?

The `java.lang.Object` class is the super class of all classes whether the class is predefined or user defined class. Object class is by default inherited whether we extend explicitly or not. There are total 11 methods of `java.lang.Object` class, out of which 5 methods are native methods.

Native methods of object class in java:

protected native `Object clone()` throws `CloneNotSupportedException`

public final native `Class getClass()`

public native `int hashCode()`

public final native `void notify()`

public final native `void notifyAll()`

Other Methods:

public boolean `equals(Object obj)`

protected void `finalize()` throws `Throwable`

public `String toString()`

public final void `wait(long timeout)` throws `InterruptedException`

public final void `wait(long timeout, int nanoseconds)` throws `InterruptedException`

public final void `wait()` throws `InterruptedException`

3. What is the need of package? Which types are allowed to declare in package?

Packages are used for:

Preventing naming conflicts.

Making searching/locating and usage of classes, interfaces, enumerations and annotations easier.

Providing controlled access: protected and default have package level access control.

Subpackages, classes, interfaces, enumerations and annotation types can be declared inside package.

4. Why we can not declare multiple public classes in single java file?

A Java source file can have only one class declared as public, we cannot put two or more public classes together in a . java file. This is because of the restriction that the file name should be same as the name of the public class with . java extension.

5. What is the difference between import and static import?

The import allows the java programmer to access classes of a package without package qualification whereas the static import feature allows to access the static members of a class without the class qualification.

6. How can we pass argument to the method by reference? Explain with example?

Java is always a pass by value; but, there are a few ways to achieve pass by reference:

1. Making a public member variable in a class

```
// my class
class my_number {
    // public member variable
    public int number;

    // default constructor
    public my_number()
    {
        number = 1;
    }
}
```

```
// driver function
class Main{
    public static void main (String [] arguments)
    {
        // creating object
        my_number object = new my_number();

        // printing before update
        System.out.println("number = " + object.number);
        // update function called.
        update(object);
        //printing after update.
        System.out.println("number = " + object.number);
    }
    // update function.
    public static void update( my_number obj ){
        // increments number variable.
        obj.number++;
    }
}
```

2. Return a value and update it

```
class Main{
    public static void main (String [] arguments)
    {
        int number = 1;
        // printing before update.
        System.out.println("number = " + number);
        // update function returns a value.
        number = update(number);
        // printing after update.
        System.out.println("number = " + number);
    }
    // update function
    public static int update( int number ){
        // increments number.
        number++;
        return number;
    }
}
```

3. Create a single element array

```
class Main{
    public static void main (String [] arguments)
    {
        // single element array.
```

```

int number[] = { 1 };

// printing before update.
System.out.println("number = " + number[0]);
//update function.
update(number);
// printing after update.
System.out.println("number = " + number[0]);
}
// update function.
public static void update( int number[] ){
    // increments the number
    number[0]++;
}
}

```

7. What is the difference between checked and unchecked exception?

There are two types of exceptions: checked exception and unchecked exception. The main difference between checked and unchecked exception is that the checked exceptions are checked at compile-time while unchecked exceptions are checked at runtime.

8. Which are the advantages and disadvantages of generics?

Advantages:

- Readability and Documentation.
- More precise indication of types.
- Less downcasting from class Object.
- Better and more precise typechecking.
- There is a potential for more efficient programs.
- Less casting - fewer boxings.

Disadvantages:

- Generics don't work with primitive types.
- We can't create Generic Arrays
- We cannot create an instance of a type parameter.
- We cannot create, catch, or throw generic types.
- Due to type erasure, you cannot use instanceof with generic types.
- We cannot declare static fields of a type parameter.

9. What is difference between Comparable and Comparator?

Comparable is meant for objects with natural ordering which means the object itself must know how it is to be ordered. For example Roll Numbers

of students. Whereas, Comparator interface sorting is done through a separate class.

Logically, Comparable interface compares "this" reference with the object specified and Comparator in Java compares two different class objects provided.

If any class implements Comparable interface in Java then collection of that object either List or Array can be sorted automatically by using Collections.sort() or Arrays.sort() method and objects will be sorted based on their natural order defined by compareTo method.

A basic differentiating feature is that using comparable we can use only one comparison. Whereas, we can write more than one custom comparators as you want for a given type, all using different interpretations of what sorting means. Like in the comparable example we could just sort by only one attribute, i.e., year but in the comparator, we were able to use different attributes like rating, name, and year as well.

10. What is difference between ArrayList and Vector? How to use ListIterator?

ArrayList and Vector both implements List interface and maintains insertion order.

ArrayList is not synchronized whereas, Vector is synchronized.

ArrayList increments 50% of current array size if the number of elements exceeds from its capacity. Vector increments 100% means doubles the array size if the total number of elements exceeds than its capacity.

ArrayList is not a legacy class. It is introduced in JDK 1.2. Vector is a legacy class.

ArrayList is fast because it is non-synchronized. Vector is slow because it is synchronized.

ArrayList uses the Iterator interface to traverse the elements. A Vector can use the Iterator interface or Enumeration interface to traverse the elements.

There are few methods of listIterator such as next(), previous(), hasNext(), hasPrevious(), nextIndex(), previousIndex(). With the help of those methods, we can traverse through list.

With the help of add(), remove(), set() methods, we can modify the list.

11. What is fail-fast and fail-safe iterator?

Iterators in java are used to iterate over the Collection objects. Fail-Fast iterators immediately throw `ConcurrentModificationException` if there is structural modification of the collection. Structural modification means adding, removing any element from collection while a thread is iterating over that collection. Iterator on `ArrayList`, `HashMap` classes are some examples of fail-fast Iterator.

Fail-Safe iterators don't throw any exceptions if a collection is structurally modified while iterating over it. This is because, they operate on the clone of the collection, not on the original collection and that's why they are called fail-safe iterators. Iterator on `CopyOnWriteArrayList`, `ConcurrentHashMap` classes are examples of fail-safe Iterator.

12. What is serialization and deserialization? What is significance of Serial Version UID?

Java serialization is the process of converting an object into a stream of bytes so we can do stuff like store it on disk or send it over the network. Deserialization is the reverse process - converting a stream of bytes into an object in memory.

`SerialVersionUID` is a unique identifier for each class, JVM uses it to compare the versions of the class ensuring that the same class was used during Serialization is loaded during Deserialization.

13. What is relation between Thread start() and run() method?

When a program calls the `start()` method, a new thread is created and then the `run()` method is executed. But if we directly call the `run()` method then no new thread will be created and `run()` method will be executed as a normal method call on the current calling thread itself and no multi-threading will take place.

14. When we should create thread by implementing Runnable and extending Thread class?

we can define a thread in the following two ways:

1. By extending Thread class
2. By implementing Runnable interface

In the first approach, Our class always extends Thread class. There is no chance of extending any other class. Hence we are missing Inheritance benefits. In the second approach, while implementing Runnable interface we can extend any other class. Hence we are able to use the benefits of Inheritance.

Because of the above reasons, implementing Runnable interface approach is recommended than extending Thread class.

15. What is functional interface? Which functional interfaces are predefined and where they are used?

An Interface that contains exactly one abstract method is known as functional interface. It can have any number of default, static methods but can contain only one abstract method. It can also declare methods of object class.

Functional Interface is also known as Single Abstract Method Interfaces or SAM Interfaces. It is a new feature in Java, which helps to achieve functional programming approach.

There are many predefined functional interfaces in java-

1. Function

A Function interface is more of a generic one that takes one argument and produces a result. This has a Single Abstract Method (SAM) apply which accepts an argument of a type T and produces a result of type R. One of the common use cases of this interface is Stream.map method.

2. Supplier

A Supplier is a simple interface which indicates that this implementation is a supplier of results. This interface, however, does not enforce any restrictions that supplier implementation needs to return a different result on each invocation.

3. Consumer

A Consumer is a functional interface that accepts a single input and returns no output.

4. Predicate

A Predicate interface represents a boolean-valued-function of an argument. This is mainly used to filter data from a Java Stream.

16. What is significance of filter(), map(), flatMap() and reduce() operations? In which scenarios they are used?

map(), filter(), reduce(), flatMap() are the operations to be performed on Streams.

The map() applies as a transformation to an element.

The filter() accumulates only elements matching a Predicate<T>.

The reduce() accumulates all elements to a single value, by using immutable.

The `flatMap()` method first flattens the input Stream of Streams to a Stream of Strings. Thereafter it works similarly to the `map()` method.

17. What is JDBC driver? Explain steps of JDBC programming,

JDBC stands for Java Database Connectivity. JDBC is a Java API to connect and execute the query with the database. It is a part of JavaSE (Java Standard Edition). JDBC API uses JDBC drivers to connect with the database.

A JDBC program comprises the following steps:

1. Allocate a Connection object, for connecting to the database server.
2. Allocate a Statement object, under the Connection created earlier, for holding a SQL command.
3. Write a SQL query and execute the query, via the Statement and Connection created.
4. Process the query result.
5. Close the Statement and Connection to free up the resources.